Our task is to Heartfailure prediction using SVM classifier

Here is the description about the dataset

age: The age of the patient.

anaemia: Presence of anemia in the patient (1 = Yes, 0 = No).

creatinine_phosphokinase: Level of the enzyme creatinine phosphokinase in the blood (measured in mcg/L).

diabetes: Whether the patient has diabetes (1 = Yes, 0 = No).

ejection_fraction: Percentage of blood leaving the heart with each contraction (measured in percentage).

high_blood_pressure: Whether the patient has high blood pressure (1 = Yes, 0 = No).

platelets: Number of platelets in the blood (measured in kiloplatelets/mL).

serum_creatinine: Level of serum creatinine in the blood (measured in mg/dL).

serum_sodium: Level of serum sodium in the blood (measured in mEq/L).

sex: The biological sex of the patient (1 = Male, 0 = Female).

smoking: Whether the patient smokes (1 = Yes, 0 = No).

time: Follow-up period for the patient (measured in days).

DEATH_EVENT: The target variable indicating if the patient died during the follow-up period (1 = Yes, 0 = No).

In [1]:

```
#Step1:importing
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,confusion_matrix,classification_r
```

```
c:\users\vamsi2001\appdata\local\programs\python\python39\lib\site-packages\num
py\_distributor_init.py:30: UserWarning: loaded more than 1 DLL from .libs:
c:\users\vamsi2001\appdata\local\programs\python\python39\lib\site-packages\num
py\.libs\libopenblas.EL2C6PLE4ZYW3ECEVIV3OXXGRN2NRFM2.gfortran-win_amd64.dll
c:\users\vamsi2001\appdata\local\programs\python\python39\lib\site-packages\num
py\.libs\libopenblas.XWYDX2IKJW2NMTWSFYNGFUWKQU3LYTCZ.gfortran-win_amd64.dll
  warnings.warn("loaded more than 1 DLL from .libs:"
```

In [2]:
```
df=pd.read_csv('HeartFailure.csv')
df.head()
```

Out[2]:

	age	anaemia	creatinine_phosphokinase	diabetes	ejection_fraction	high_blood_pressure	plate
0	75.0	0	582	0	20	1	26500
1	55.0	0	7861	0	38	0	26335
2	65.0	0	146	0	20	0	16200
3	50.0	1	111	0	20	0	21000
4	65.0	1	160	1	20	0	32700

In [3]:
```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 299 entries, 0 to 298
Data columns (total 13 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   age                       299 non-null    float64
 1   anaemia                   299 non-null    int64
 2   creatinine_phosphokinase  299 non-null    int64
 3   diabetes                  299 non-null    int64
 4   ejection_fraction         299 non-null    int64
 5   high_blood_pressure       299 non-null    int64
 6   platelets                 299 non-null    float64
 7   serum_creatinine          299 non-null    float64
 8   serum_sodium              299 non-null    int64
 9   sex                       299 non-null    int64
 10  smoking                   299 non-null    int64
 11  time                      299 non-null    int64
 12  DEATH_EVENT               299 non-null    int64
dtypes: float64(3), int64(10)
memory usage: 30.5 KB
```

Out[3]:

	age	anaemia	creatinine_phosphokinase	diabetes	ejection_fraction	high_blood_
count	299.000000	299.000000	299.000000	299.000000	299.000000	29
mean	60.833893	0.431438	581.839465	0.418060	38.083612	
std	11.894809	0.496107	970.287881	0.494067	11.834841	
min	40.000000	0.000000	23.000000	0.000000	14.000000	
25%	51.000000	0.000000	116.500000	0.000000	30.000000	
50%	60.000000	0.000000	250.000000	0.000000	38.000000	
75%	70.000000	1.000000	582.000000	1.000000	45.000000	
max	95.000000	1.000000	7861.000000	1.000000	80.000000	

In [ ]:

```

```

In [4]:

```
df.columns
```

Out[4]:

```
Index(['age', 'anaemia', 'creatinine_phosphokinase', 'diabetes',
       'ejection_fraction', 'high_blood_pressure', 'platelets',
       'serum_creatinine', 'serum_sodium', 'sex', 'smoking', 'time',
       'DEATH_EVENT'],
      dtype='object')
```

In [5]:

```
cat_cols=['anaemia','diabetes','high_blood_pressure','sex','smoking','DEATH_
for i in cat_cols:
    df[i]=df[i].astype('category')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 299 entries, 0 to 298
Data columns (total 13 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   age                       299 non-null    float64
 1   anaemia                   299 non-null    category
 2   creatinine_phosphokinase  299 non-null    int64
 3   diabetes                  299 non-null    category
 4   ejection_fraction         299 non-null    int64
 5   high_blood_pressure       299 non-null    category
 6   platelets                 299 non-null    float64
 7   serum_creatinine          299 non-null    float64
 8   serum_sodium              299 non-null    int64
 9   sex                       299 non-null    category
 10  smoking                   299 non-null    category
 11  time                      299 non-null    int64
 12  DEATH_EVENT               299 non-null    category
dtypes: category(6), float64(3), int64(4)
memory usage: 19.0 KB
```

In [7]:

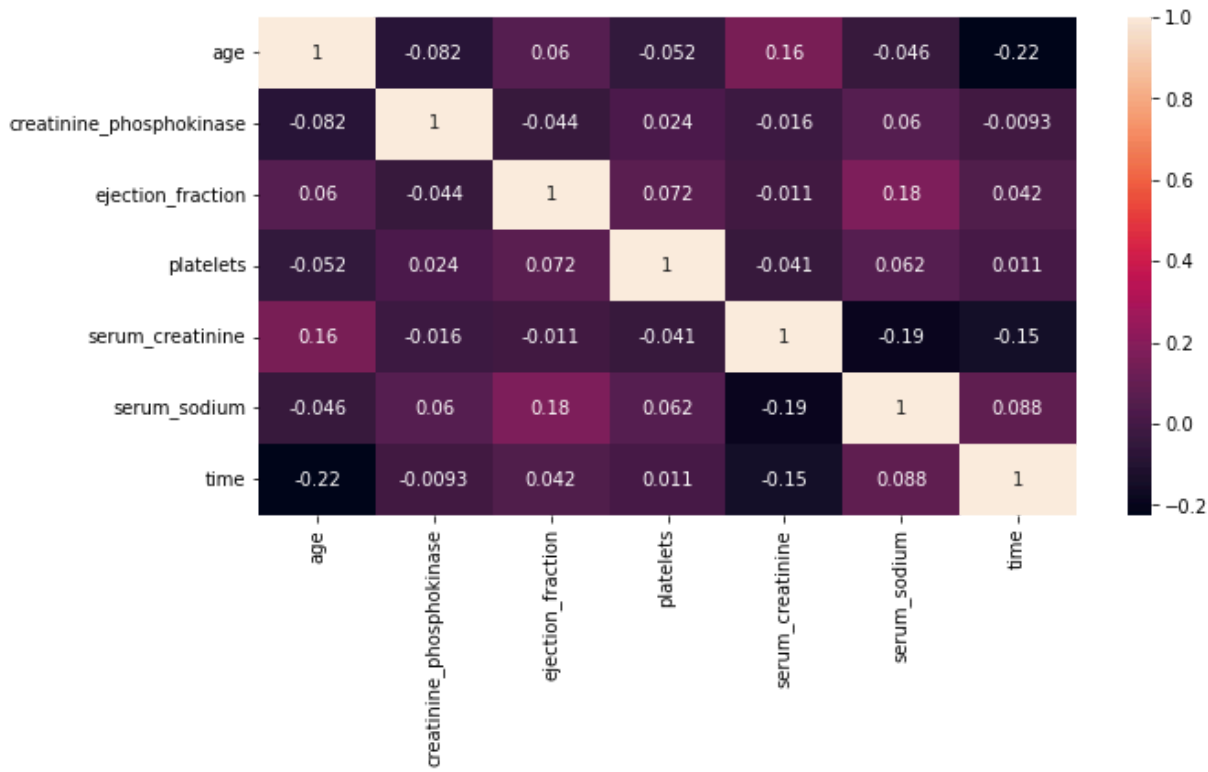
```
df.describe()
```

Out[7]:

	age	creatinine_phosphokinase	ejection_fraction	platelets	serum_creatinine	se
count	299.000000	299.000000	299.000000	299.000000	299.00000	
mean	60.833893	581.839465	38.083612	263358.029264	1.39388	
std	11.894809	970.287881	11.834841	97804.236869	1.03451	
min	40.000000	23.000000	14.000000	25100.000000	0.50000	
25%	51.000000	116.500000	30.000000	212500.000000	0.90000	
50%	60.000000	250.000000	38.000000	262000.000000	1.10000	
75%	70.000000	582.000000	45.000000	303500.000000	1.40000	
max	95.000000	7861.000000	80.000000	850000.000000	9.40000	

In [8]:
```
plt.figure(figsize=(10,5))
corrmat=df.corr()
sns.heatmap(corrmat, annot=True)
```

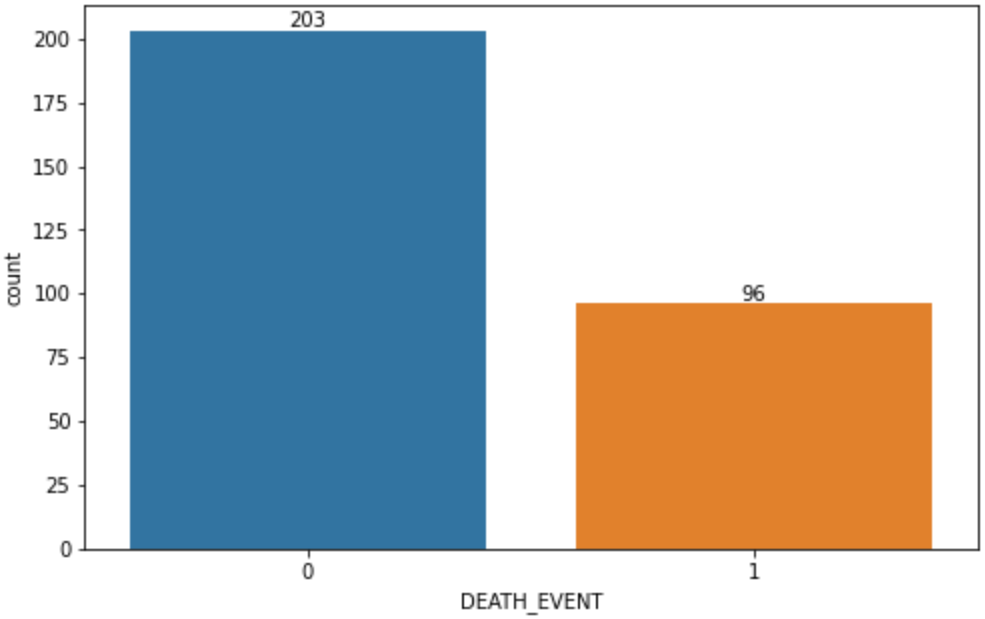
Out[8]: <AxesSubplot:>



In [9]:
```
event_counts=df['DEATH_EVENT'].value_counts()

plt.figure(figsize=(8,5))
ax=sns.countplot(x=df['DEATH_EVENT'])
ax.bar_label(ax.containers[0])
```

Out[9]: [Text(0, 0, '203'), Text(0, 0, '96')]



In [9]:
```
X=df.drop(columns='DEATH_EVENT', axis=1)
Y=df['DEATH_EVENT']
```

In [10]:
```
cols=list(X.columns)
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
X_scaled_df=pd.DataFrame(X_scaled, columns=cols)
X_scaled_df.describe().T
```

Out[10]:

	count	mean	std	min	25%	50%	75%
age	299.0	5.703353e-16	1.001676	-1.754448	-0.828124	-0.070223	0.771889
anaemia	299.0	1.009969e-16	1.001676	-0.871105	-0.871105	-0.871105	1.147968
creatinine_phosphokinase	299.0	0.000000e+00	1.001676	-0.576918	-0.480393	-0.342574	0.000166
diabetes	299.0	9.060014e-17	1.001676	-0.847579	-0.847579	-0.847579	1.179830
ejection_fraction	299.0	-3.267546e-17	1.001676	-2.038387	-0.684180	-0.007077	0.585389
high_blood_pressure	299.0	0.000000e+00	1.001676	-0.735688	-0.735688	-0.735688	1.359272
platelets	299.0	7.723291e-17	1.001676	-2.440155	-0.520870	-0.013908	0.411120
serum_creatinine	299.0	1.425838e-16	1.001676	-0.865509	-0.478205	-0.284552	0.005926
serum_sodium	299.0	-8.673849e-16	1.001676	-5.363206	-0.595996	0.085034	0.766064
sex	299.0	-8.911489e-18	1.001676	-1.359272	-1.359272	0.735688	0.735688
smoking	299.0	-1.188199e-17	1.001676	-0.687682	-0.687682	-0.687682	1.454161
time	299.0	-1.901118e-16	1.001676	-1.629502	-0.739000	-0.196954	0.938759

In [11]:
```
X_scaled_df.head()
```

Out[11]:

	age	anaemia	creatinine_phosphokinase	diabetes	ejection_fraction	high_blood_pressure
0	1.192945	-0.871105	0.000166	-0.847579	-1.530560	1.359272
1	-0.491279	-0.871105	7.514640	-0.847579	-0.007077	-0.735688
2	0.350833	-0.871105	-0.449939	-0.847579	-1.530560	-0.735688
3	-0.912335	1.147968	-0.486071	-0.847579	-1.530560	-0.735688
4	0.350833	1.147968	-0.435486	1.179830	-1.530560	-0.735688

In [12]:
```

x_train, x_test, y_train, y_test = train_test_split(X_scaled_df, Y, test_siz
```

In [13]:
```
from sklearn.svm import SVC
modelsvm=SVC(kernel='sigmoid').fit(x_train, y_train)
```

In [14]:
```
y_pred=modelsvm.predict(x_test)
y_pred
```

Out[14]:
```
array([0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1,
       1, 0], dtype=int64)
```

In [15]:
```
print(accuracy_score(y_test, y_pred))

```

```
0.7444444444444445
```

In [20]:
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.76      0.91      0.83        53
           1       0.81      0.59      0.69        37

    accuracy                           0.78        90
   macro avg       0.79      0.75      0.76        90
weighted avg       0.78      0.78      0.77        90
```

GridSearchCV

GridSearchCV is a hyperparameter tuning technique provided by scikit learn.

It exhaustively searches through all possible combinations of hyperparameter values you specify.

For each combination, it evaluates model performance using crossvalidation.

Finally, it picks the combination that gives the best performance.

C: Regularization parameter which controls the trade-off between correct classification and margin size.

gamma: Kernel coefficient (for rbf, poly, sigmoid). It controls how far the influence of a single training example reaches.

kernel: The type of kernel function to use (linear, rbf, or poly).

This creates 4 × 5 × 3 = 60 total combinations to try

In [17]:
```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto', 0.01, 0.1, 1],
    'kernel': ['rbf', 'linear', 'poly']
}

grid = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy')
grid.fit(x_train, y_train)

```

Out[17]:
```
GridSearchCV(cv=5, estimator=SVC(),
             param_grid={'C': [0.1, 1, 10, 100],
                         'gamma': ['scale', 'auto', 0.01, 0.1, 1],
                         'kernel': ['rbf', 'linear', 'poly']},
             scoring='accuracy')
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [19]:
```
print("Best Parameters:", grid.best_params_)

best_model = grid.best_estimator_
y_pred = best_model.predict(x_test)

print("Accuracy after tuning:", accuracy_score(y_test, y_pred))
```

```
Best Parameters: {'C': 10, 'gamma': 0.01, 'kernel': 'rbf'}
Accuracy after tuning: 0.7777777777777778
```

RandomizedSearchCV

In [ ]:
```

```

In [22]:

```
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    'C': np.logspace(-2, 2, 10),
    'kernel': ['linear', 'rbf', 'poly'],
    'gamma': ['scale', 'auto']
}

random_search = RandomizedSearchCV(SVC(), param_distributions=param_dist,
                                   n_iter=10, cv=3, scoring='accuracy', rand
random_search.fit(x_train, y_train)
best_model = random_search.best_estimator_
y_pred = best_model.predict(x_test)

print("Best Parameters (RandomSearch):", random_search.best_params_)
print("Best Score (RandomSearch):", random_search.best_score_)

```

```
Fitting 3 folds for each of 10 candidates, totalling 30 fits
Best Parameters (RandomSearch): {'kernel': 'linear', 'gamma': 'scale', 'C': 0.0
774263682681127}
Best Score (RandomSearch): 0.798895790200138
```

In [ ]:

```

```

Type *Markdown* and LaTeX: α^2

In [ ]:

```

```

In [24]:

```
import seaborn as sns
data=sns.load_dataset('titanic')
data.head()
```

Out[24]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	dec
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	Nal
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	(
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Nal
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	(
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Nal

In [25]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

In [26]:
```
data.isnull().sum()
```

Out[26]:
```
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
```

In [27]:
```
#handling missing
mean=data['age'].mean()
data['age']=data['age'].fillna(data['age'].mean())
```

In [28]:
```
data.deck.value_counts()
```

Out[28]:
```
C    59
B    47
D    33
E    32
A    15
F    13
G     4
Name: deck, dtype: int64
```

In [29]:
```
data.embarked.value_counts()
```

Out[29]:
```
S    644
C    168
Q     77
Name: embarked, dtype: int64
```

In [30]:
```
data['deck']=data['deck'].fillna('C')
data['embarked']=data['embarked'].fillna('S')
```

In [31]:
```
data=data.drop('embark_town',axis=1)
```

In [32]:
```
data.isnull().sum()
```

Out[32]:
```
survived      0
pclass        0
sex           0
age           0
sibsp         0
parch         0
fare          0
embarked      0
class         0
who           0
adult_male    0
deck          0
alive         0
alone         0
dtype: int64
```

In [33]:
```
data.head()
```

Out[33]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	dec
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	(
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	(
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	(
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	(
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	(

In [34]:
```
data.columns
```

Out[34]:
```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
       'embarked', 'class', 'who', 'adult_male', 'deck', 'alive', 'alone'],
      dtype='object')
```

In [39]:
```
from sklearn.preprocessing import LabelEncoder,StandardScaler
cat=['sex','embarked','class','who','adult_male', 'deck', 'alive', 'alone']
lb=LabelEncoder()
for i in cat:
    data[i]=lb.fit_transform(data[i])
data.head()
```

Out[39]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	ali
0	0	3	1	22.0	1	0	7.2500	2	2	1	1	2	
1	1	1	0	38.0	1	0	71.2833	0	0	2	0	2	
2	1	3	0	26.0	0	0	7.9250	2	2	2	0	2	
3	1	1	0	35.0	1	0	53.1000	2	0	2	0	2	
4	0	3	1	35.0	0	0	8.0500	2	2	1	1	2	

In [47]:
```
x=data.drop('survived',axis=1)
y=data['survived']
```

In [45]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_stat
```

In [64]:
```
from sklearn.svm import SVC
clf=SVC(kernel='sigmoid')
clf.fit(x_train,y_train)
```

Out[64]:
```
SVC(kernel='sigmoid')
```

In [65]:
```
y_pred=clf.predict(x_test)
y_pred
```

Out[65]:
```
array([1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1,
       1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1,
       0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0,
       1, 1, 0], dtype=int64)
```

In [66]:
```
from sklearn.metrics import accuracy_score,classification_report,confusion_m
accuracy_score(y_test,y_pred)
```

Out[66]:
```
0.9888268156424581
```

In [67]:
```
print(classification_report(y_test,y_pred))
```

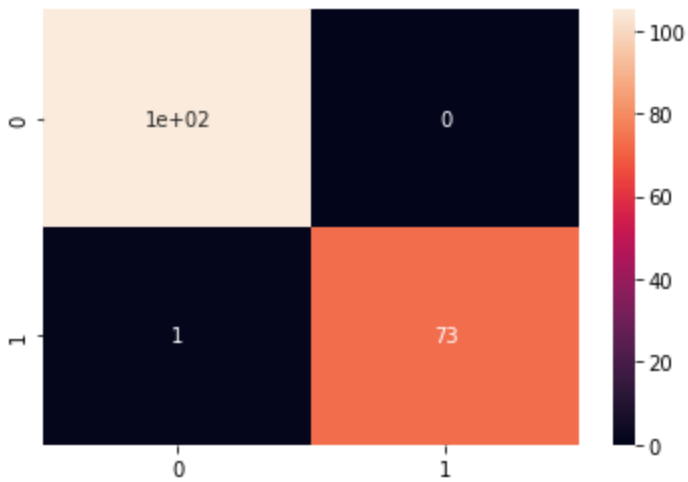
```
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       105
           1       1.00      0.97      0.99        74

    accuracy                           0.99       179
   macro avg       0.99      0.99      0.99       179
weighted avg       0.99      0.99      0.99       179
```

In [55]:
```
sns.heatmap(data=confusion_matrix(y_test,y_pred),annot=True)
```

Out[55]: <AxesSubplot:>



In [ ]:
```

```